实验三　mybatis 动态 SQL

一、实验目的及任务

目的：掌握利用 mybatis 进行 java 对象数据存取，并使用动态 SQL 编写 SQL 语句的方法。
任务：
（1）完成对角色表的增删改查操作。
（2）使用动态 SQL 中的 if+set 和 trim+if 编写修改角色 update 操作。
（3）使用 if+trim 实现根据角色名称模糊查询角色信息列表，并进行分页显示。

二、实验环境及条件

主机操作系统为 Windows11，Tomcat8，JDK8 及以上,IDEA。

三、实验实施步骤

0 实验步骤（略）不用写。

1 实验结果：

（1） 包括所有自己编写的代码文件，接口类.java，Mapper.XML

1. cn.smbms.dao.role.RoleMapper.java

```
package cn.smbms.dao.role;

import cn.smbms.pojo.Role;
import org.apache.ibatis.annotations.Param;

import java.util.List;

public interface RoleMapper {
    /**
     * 根据角色名称查询角色信息列表
     * @param roleName
     * @return
     */
```

```
    public List<Role> getRoleListByRoleName(@Param("roleName") String
roleName,
                                        @Param("form") Integer from,
                                        @Param("pageSize") Integer
pageSize);
    public int add(Role role);

    /**
     * 根据角色id修改角色信息
     * @Param role
     * @return
     */
    public int modify(Role role);
    public int modify2(Role role);

    /**
     *根据角色id查询该角色下是否有用户信息
     * @param roleId
     * @return
     */
    public int getUserCountByRoleId(@Param("userRole") Integer roleId);

    /**
     *根据角色id删除角色信息
     * @param roleId
     * @return
     */
    public int deleteRoleById(@Param("id") Integer roleId);

    /**
     * 添加一个一次删除多个角色
     * @param roleIds
     * @return
     */
    public int deleteRoles(@Param("roleIds") Integer[] roleIds);

}
```

2. cn.smbms.dao.role.RoleMapper.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE mapper
        PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
        "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
<mapper namespace="cn.smbms.dao.role.RoleMapper">
```

```
    <!-- 根据角色名称查询角色信息列表（模糊查询），并按照 creationDate 降序排列，使用 limit 关键字进行分页 -->
    <select id="getRoleListByRoleName" resultType="cn.smbms.pojo.Role">
        SELECT * FROM role
        <where>
            <if test="roleName != null and roleName != ''">
                AND role_name LIKE CONCAT('%', #{roleName}, '%')
            </if>
        </where>
        ORDER BY creationDate DESC
        LIMIT #{from}, #{pageSize}
    </select>

    <!-- 根据角色 id 查询该角色下是否有用户信息，返回该角色 id 下的用户数量 -->
    <select id="getUserCountByRoleId" resultType="java.lang.Integer">
        SELECT COUNT(*) FROM user
        WHERE role_id = #{roleId}
    </select>
    <!-- insert 元素 -->
    <insert id="add" parameterType="cn.smbms.pojo.Role">
        INSERT INTO smbms_role (roleCode, roleName, createdBy, creationDate)
        VALUES (#{roleCode}, #{roleName}, #{createdBy}, #{creationDate})
    </insert>

    <!-- update 元素 - set+if 方式 -->
    <update id="modify" parameterType="cn.smbms.pojo.Role">
        UPDATE smbms_role
        <set>
            <if test="roleCode != null">roleCode = #{roleCode},</if>
            <if test="roleName != null">roleName = #{roleName},</if>
            <if test="modifyBy != null">modifyBy = #{modifyBy},</if>
            <if test="modifyDate != null">modifyDate = #{modifyDate}</if>
        </set>
        WHERE id = #{id}
    </update>

    <!-- update 元素 - trim+if 方式 -->
    <update id="modify2" parameterType="cn.smbms.pojo.Role">
        UPDATE smbms_role
        <trim prefix="SET" suffixOverrides=",">
            <if test="roleCode != null">roleCode = #{roleCode},</if>
            <if test="roleName != null">roleName = #{roleName},</if>
```

```
            <if test="modifyBy != null">modifyBy = #{modifyBy},</if>
            <if test="modifyDate != null">modifyDate = #{modifyDate}</if>
        </trim>
        WHERE id = #{id}
    </update>

    <!-- delete 元素 -->
    <delete id="deleteRoles" parameterType="java.util.List">
        DELETE FROM smbms_role WHERE id IN
        <foreach collection="roleIds" item="item" open="(" separator="," 
close=")">
            #{item}
        </foreach>
    </delete>
</mapper>
```

3． dao/role/RoleMapperTest.java 测试类

```
package dao.role;

import cn.smbms.dao.role.RoleMapper;
import cn.smbms.pojo.Role;
import cn.smbms.utils.MyBatisUtil;
import org.apache.ibatis.session.SqlSession;
import org.junit.Test;

import java.util.Date;

public class RoleMapperTest {
    @Test
    public void testAdd(){
        System.out.println("testAdd !===================");
        SqlSession sqlSession = null;
        int count = 0;
        try {
            sqlSession = MyBatisUtil.createSqlSession();
            Role role = new Role();
            role.setId(521);
            role.setRoleCode("测试添加角色");
            role.setRoleName("测试角色姓名");
            role.setCreatedBy(1);
            role.setCreationDate(new Date());
            count = sqlSession.getMapper(RoleMapper.class).add(role);
            //模拟异常，进行回滚
            //int i = 2/0;
```

```
            sqlSession.commit();
        } catch (Exception e) {
            // TODO: handle exception
            e.printStackTrace();
            sqlSession.rollback();
            count = 0;
        }finally{
            MyBatisUtil.closeSqlSession(sqlSession);
        }
        System.out.println("testAdd count: " + count);
    }

    @Test
    public void testModify(){
        System.out.println("testModify !===================");
        SqlSession sqlSession = null;
        int count = 0;
        try {
            sqlSession = MyBatisUtil.createSqlSession();
            Role role = new Role();
            role.setId(4);
            role.setRoleCode("测试修改角色");
            role.setRoleName("测试修改角色姓名");
            role.setModifyBy(1);
            role.setModifyDate(new Date());
            count = sqlSession.getMapper(RoleMapper.class).modify(role);
            //模拟异常，进行回滚
            //int i = 2/0;
            sqlSession.commit();
        } catch (Exception e) {
            // TODO: handle exception
            e.printStackTrace();
            sqlSession.rollback();
            count = 0;
        }finally{
            MyBatisUtil.closeSqlSession(sqlSession);
        }
        System.out.println("testModify count: " + count);
    }

    @Test
    public void testModify2(){
        System.out.println("testModify !===================");
        SqlSession sqlSession = null;
```

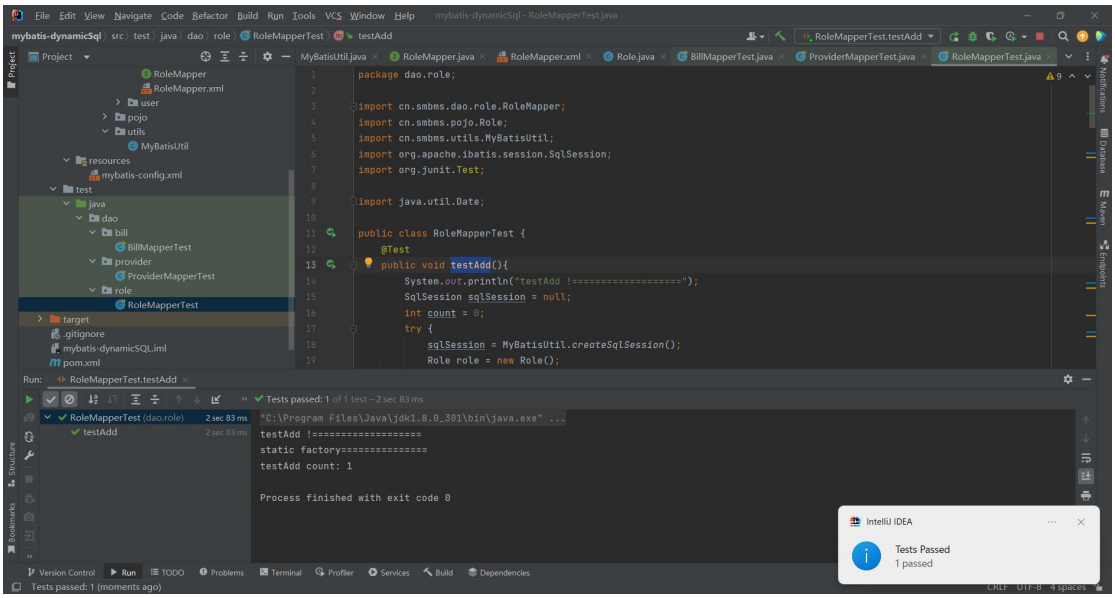
```
        int count = 0;
        try {
            sqlSession = MyBatisUtil.createSqlSession();
            Role role = new Role();
            role.setId(5);
            role.setRoleCode("测试修改角色");
            role.setRoleName("测试修改角色姓名");
            role.setModifyBy(1);
            role.setModifyDate(new Date());
            count = sqlSession.getMapper(RoleMapper.class).modify2(role);
            //模拟异常，进行回滚
            //int i = 2/0;
            sqlSession.commit();
        } catch (Exception e) {
            // TODO: handle exception
            e.printStackTrace();
            sqlSession.rollback();
            count = 0;
        }finally{
            MyBatisUtil.closeSqlSession(sqlSession);
        }
        System.out.println("testModify count: " + count);
    }

    @Test
    public void testDeleteRolesById() {
        System.out.println("testDeleteRolesById !===================");
        SqlSession sqlSession = null;
        Integer delId[] = {4,5};
        int count = 0;
        try {
            sqlSession = MyBatisUtil.createSqlSession();
            count =
sqlSession.getMapper(RoleMapper.class).deleteRoles(delId);
            sqlSession.commit();
        } catch (Exception e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
            sqlSession.rollback();
            count = 0;
        }finally{
            MyBatisUtil.closeSqlSession(sqlSession);
        }
        System.out.println("testDeleteRolesById count
```

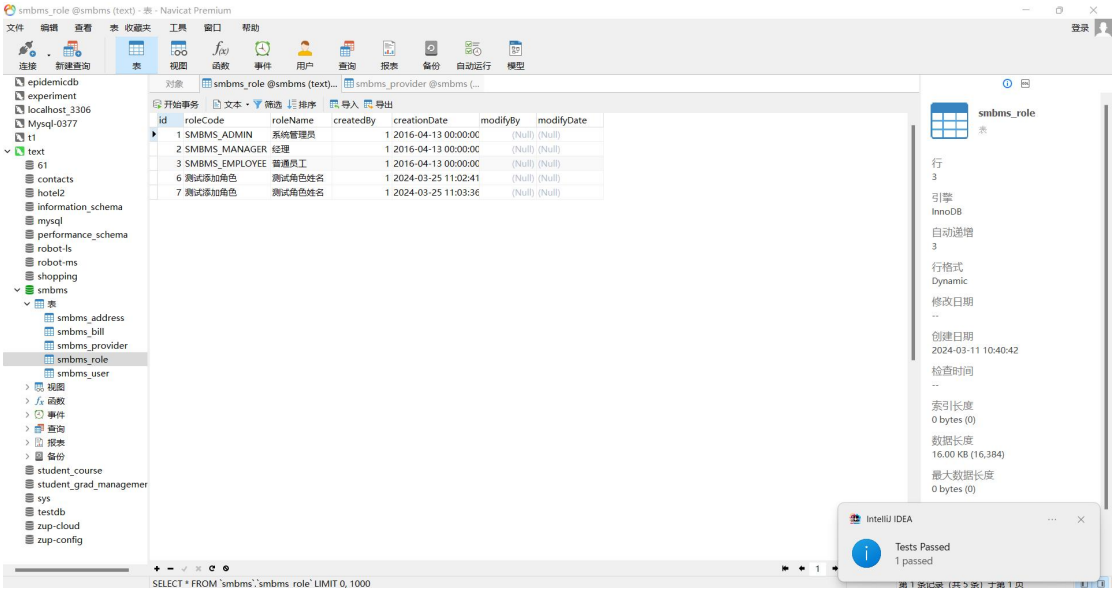
```
==================== " + count);
    }
}
```

（2） 控制台运行结果截图。

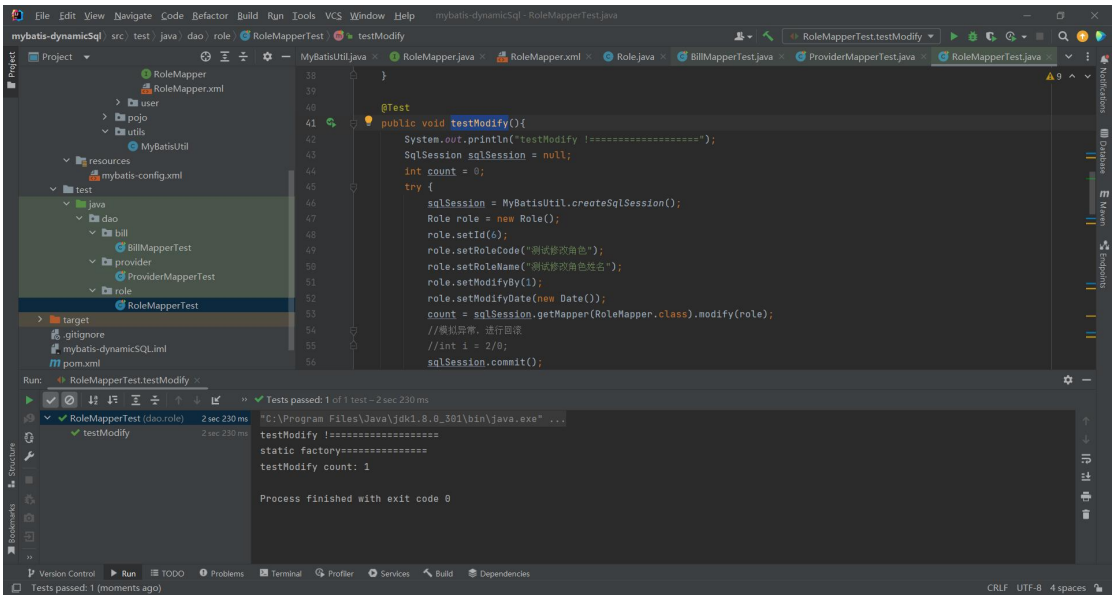
1. add 添加方法测试

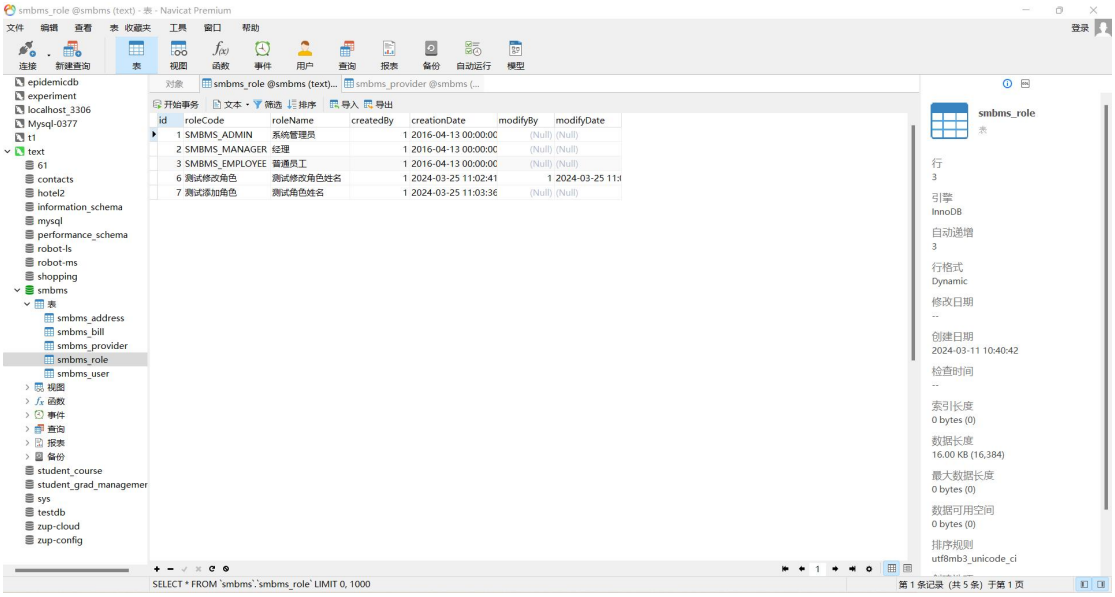


增加两个用户

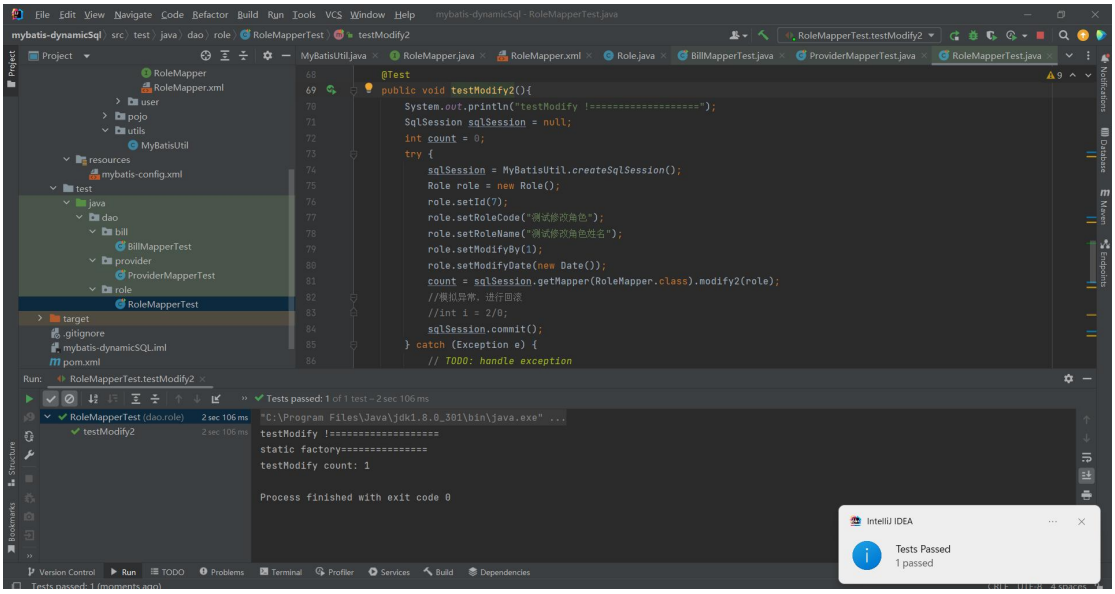


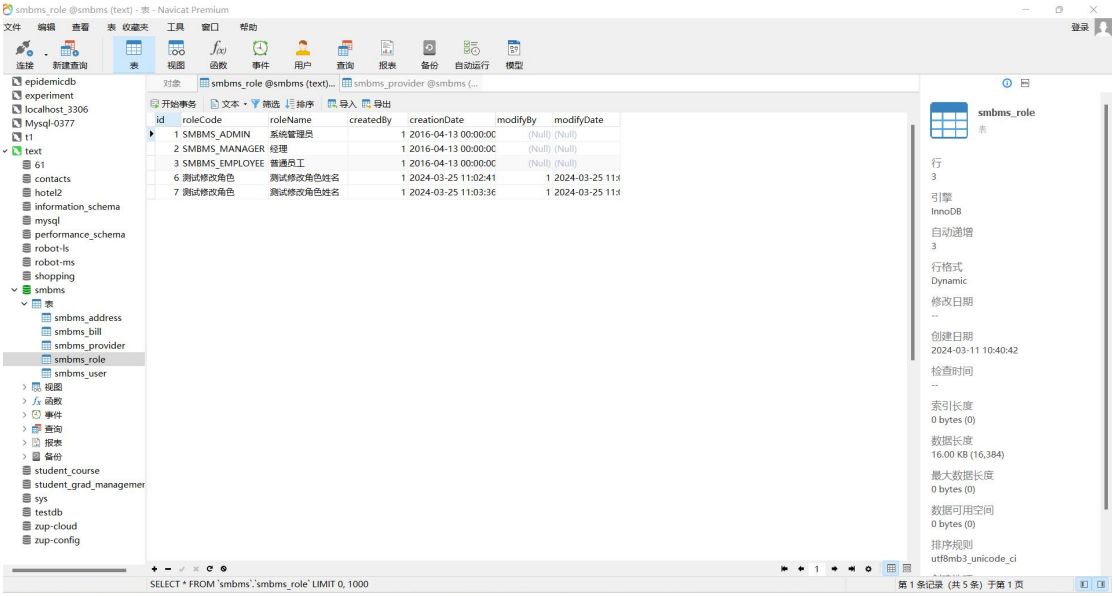
2. modify 修改方法测试



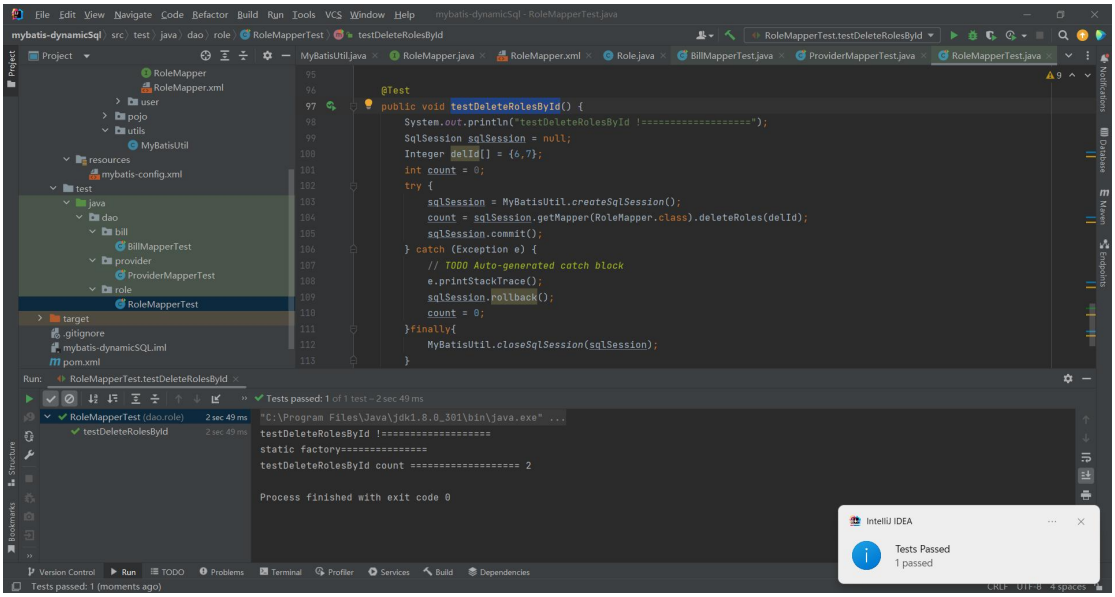


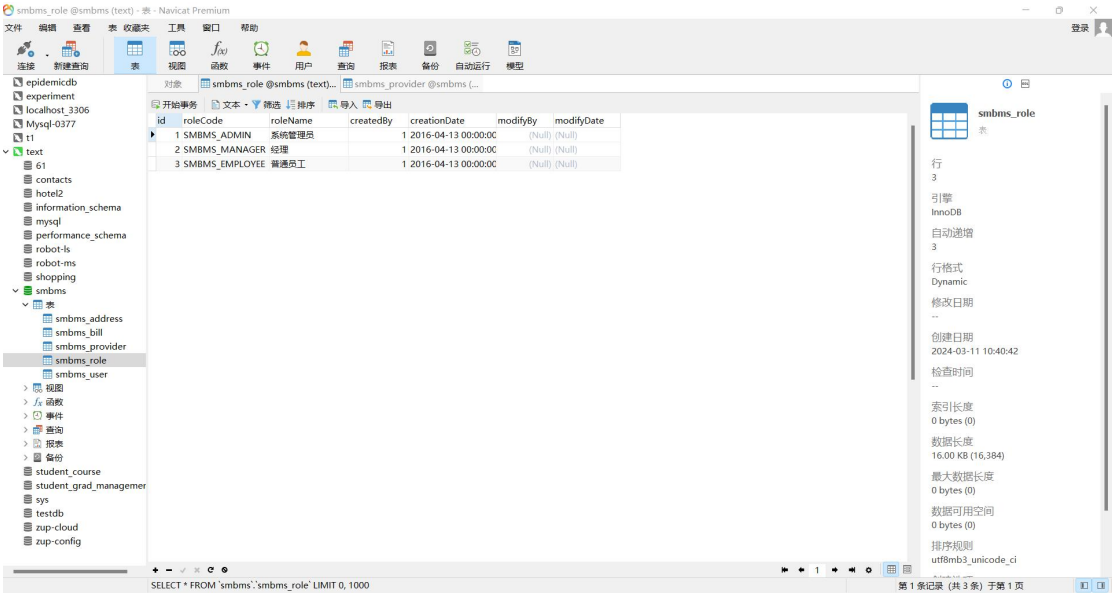
3. modify2 修改方法测试





4. 通过 id 删除多个 role 的方法测试





四、 你遇到了什么问题，如何解决的。

本次实验中，掌握利用 mybatis 进行 java 对象数据存取，并使用动态 SQL 编写 SQL 语句的方法。完成对角色表的增删改查操作。使用动态 SQL 中的 if+set 和 trim+if 编写修改角色 update 操作。使用 if+trim 实现根据角色名称模糊查询角色信息列表，并进行分页显示。编写测试类时，要把 xml 映射文件与接口对应起来，入参也要对应起来。